



Understand key concepts

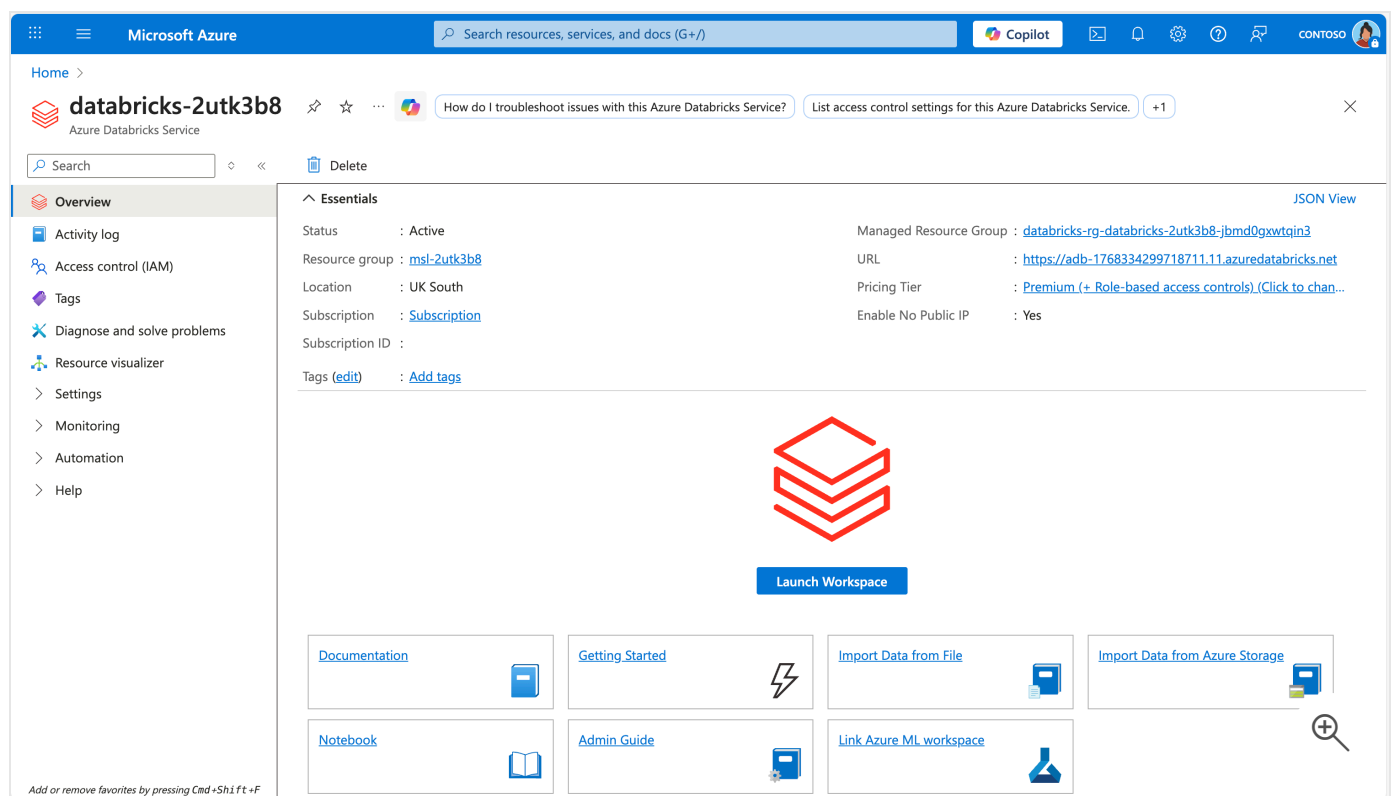
7 minutes

Azure Databricks is a single service platform with multiple technologies that enable working with data at scale. When using Azure Databricks, there are some key concepts to understand.

Workspaces

A **workspace** in Azure Databricks is a secure, collaborative environment where you can access and organize all Databricks assets, such as notebooks, clusters, jobs, libraries, dashboards, and experiments.

You can open an Azure Databricks Workspace from the Azure portal, by selecting **Launch Workspace**.



It provides a web-based **user interface (UI)** as well as REST APIs for managing resources and workflows. Workspaces can be structured into folders to organize projects, data pipelines, or team assets, and permissions can be applied at different levels to control access. They support **collaboration** by allowing multiple users—such as data engineers, analysts, and data scientists—to work together on shared notebooks, track experiments, and manage dependencies.

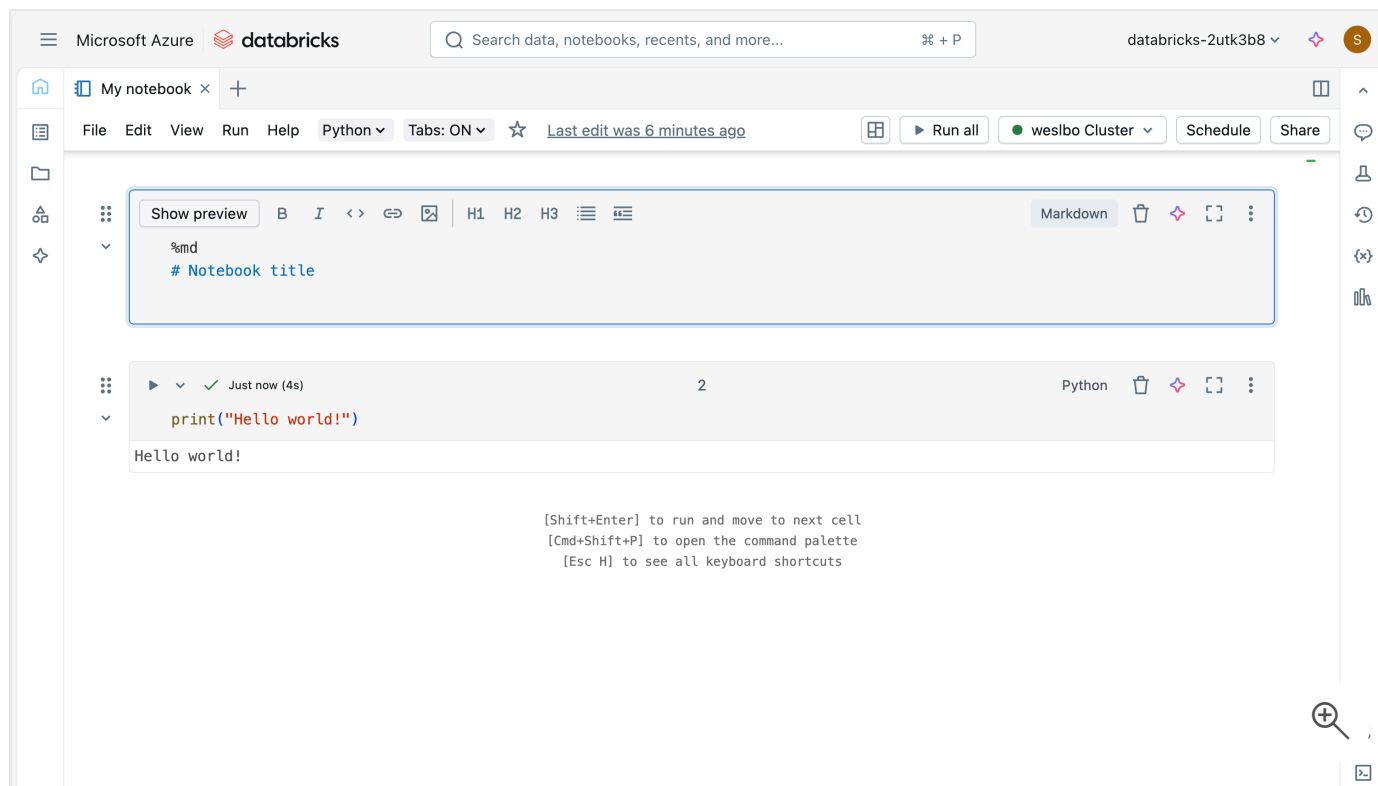
In addition, workspaces are tied to **Unity Catalog** (when enabled) for centralized data governance, ensuring secure access to data across the organization. Each workspace is also linked to an **underlying Azure resource group** (including a managed resource group) that holds the compute, networking, and storage resources Databricks uses behind the scenes.

Notebooks

Databricks notebooks are interactive, web-based documents that combine **runnable code, visualizations, and narrative text** in a single environment. They support multiple languages—such as Python, R, Scala, and SQL—and allow users to switch between languages within the same notebook using *magic commands*. This flexibility makes notebooks well-suited for **exploratory data analysis, data visualization, machine learning experiments, and building complex data pipelines**.

Notebooks are also designed for **collaboration**: multiple users can edit and run cells simultaneously, add comments, and share insights in real time. They integrate tightly with Databricks clusters, enabling users to process large datasets efficiently, and can connect to external data sources through **Unity Catalog** for governed data access. In addition, notebooks can be version-controlled, scheduled as jobs, or exported for sharing outside the platform, making them central to both **ad-hoc exploration** and **production-grade workflows**.

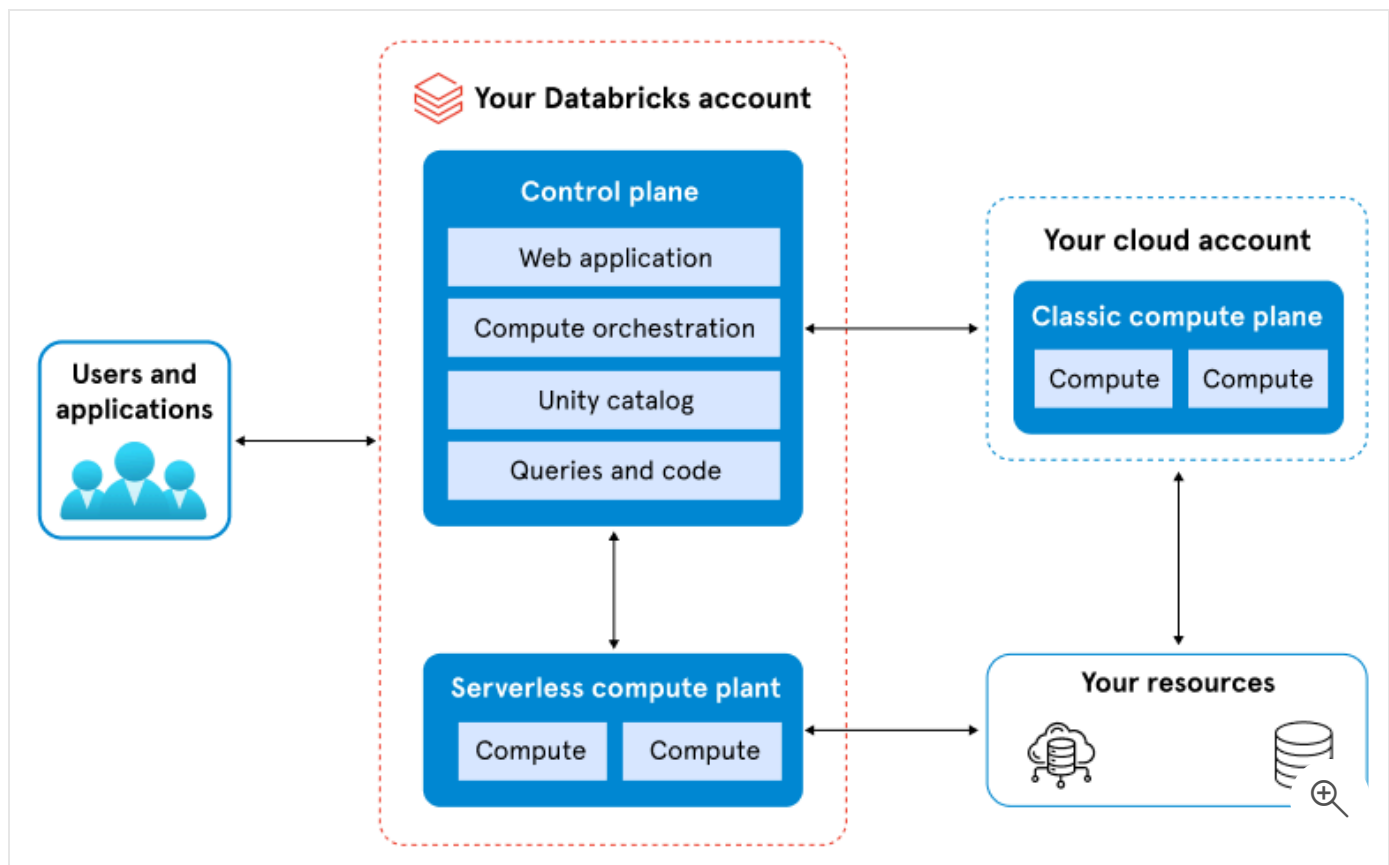
Notebooks contain a collection of two types of cells: **code cells** and **Markdown cells**. Code cells contain runnable code. Markdown cells contain Markdown code that renders as text and graphics. You can **run** a single cell, a group of cells, or the whole notebook.



Clusters

Azure Databricks leverages a two-layer architecture:

- **Control Plane:** this internal layer, managed by Microsoft, handles backend services specific to your Azure Databricks account.
- **Compute plane:** this is the external layer that processes the data and lives in your Azure Subscription.



Clusters are the core computational engines in Azure Databricks. They provide the processing power required to run data engineering, data science, and analytics tasks. Each cluster consists of a **driver node**, which coordinates execution, and one or more **worker nodes**, which handle the distributed computations. Clusters can be created manually with fixed resources or set to **auto-scale**, allowing Databricks to add or remove worker nodes depending on workload demand. This flexibility ensures efficient resource use and cost control.

Azure Databricks Compute offers a broad set of compute options available for different workload types:

- **Serverless compute:** Fully managed, on-demand compute that automatically scales up or down to meet workload needs. Ideal for teams that want fast startup times, minimal management overhead, and elastic scaling.
- **Classic compute:** User-provisioned and configured clusters that offer full control over compute settings, such as VM sizes, libraries, and runtime versions. Best for specialized workloads that require customization or consistent performance.
- **SQL warehouses:** Compute resources optimized for SQL-based analytics and BI queries. SQL warehouses can be provisioned as either **serverless** (elastic, managed) or **classic** (user-configured) depending on governance and performance requirements.

This allows you to tailor compute to specific needs—from exploratory analysis in notebooks, to large-scale ETL pipelines, to high-performance dashboards and reporting.

Databricks Runtime

The **Databricks Runtime** is a set of customized builds of **Apache Spark** that include performance improvements and additional libraries. These runtimes make it easier to handle tasks such as **machine learning**, **graph processing**, and **genomics**, while still supporting general data processing and analytics.

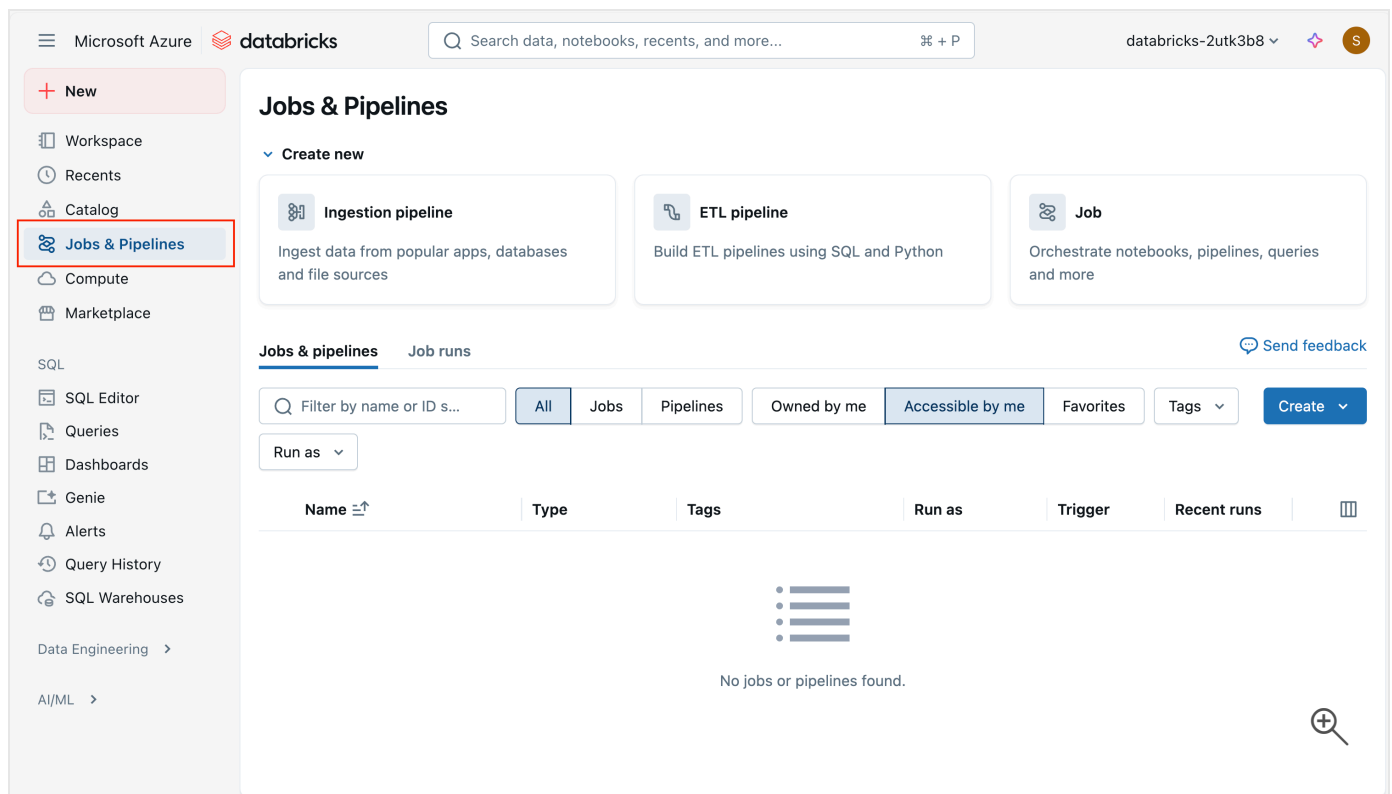
Databricks provides multiple runtime versions, including **long-term support (LTS)** releases. Each release specifies the underlying Apache Spark version, its release date, and when support will end. Over time, older runtime versions follow a lifecycle:

- **Legacy** – available but no longer recommended.
- **Deprecated** – marked for removal in a future release.
- **End of Support (EoS)** – no further patches or fixes are provided.
- **End of Life (EoL)** – retired and no longer available.

If a maintenance update is released for a runtime version you're using, you can apply it by **restarting your cluster**.

Lakeflow Jobs

Lakeflow Jobs provide workflow automation and orchestration in Azure Databricks, making it possible to reliably schedule, coordinate, and run data processing tasks. Instead of running code manually, you can use jobs to automate repetitive or production-grade workloads such as ETL pipelines, machine learning training, or dashboard refreshes.



A job in Databricks is essentially a container for one or more **tasks**. Tasks define the work to be done—for example, running a notebook, executing a Spark job, calling external code, ...

Jobs can be triggered in different ways:

- On a schedule (for example, every night at midnight).
- In response to an event.
- Manually, when needed.

Because they're repeatable and managed, jobs are critical for **production workloads**. They ensure that data pipelines run consistently, ML models are trained and deployed in a controlled manner, and downstream systems receive updated, accurate data.

Delta Lake

Delta Lake is an open-source storage framework that improves the reliability and scalability of data lakes by adding transactional features on top of cloud object storage, such as **Azure Data Lake Storage**. Traditional data lakes can suffer from issues like inconsistent data, partial writes, or difficulties managing concurrent access. Delta Lake addresses these problems by supporting:

- **ACID transactions** (atomicity, consistency, isolation, durability) for reliable reads and writes.

- **Scalable metadata handling** so tables can grow to billions of files without performance loss.
- **Data versioning and rollback**, enabling time travel queries and recovery of previous states.
- **Unified batch and streaming processing**, so the same table can handle real-time ingestion and historical batch loads.

On top of this foundation, **Delta tables** provide a familiar table abstraction that makes it easier to work with structured data using **SQL queries** or the **DataFrame API**. Delta tables are the **default table format in Azure Databricks**, ensuring that new data is stored with transactional guarantees by default.

Databricks SQL

Databricks SQL brings **data warehousing capabilities** to the Databricks Lakehouse, allowing analysts and business users to query and visualize data stored in open formats directly in the data lake. It supports **ANSI SQL**, so anyone familiar with SQL can run queries, build reports, and create dashboards without needing to learn new languages or tools.

Databricks SQL is available only in the **Premium tier** of Azure Databricks. It includes:

- A **SQL editor** for writing and running queries.
- **Dashboards and visualization tools** for sharing insights.
- Integration with external BI and analytics tools.

SQL Warehouses

All Databricks SQL queries run on **SQL warehouses** (formerly called SQL endpoints), which are scalable compute resources decoupled from storage. Different warehouse types are available depending on performance, cost, and management needs:

- **Serverless SQL Warehouses**
 - **Instant and elastic compute** with fast startup and autoscaling.
 - **Low management overhead** since Databricks handles capacity, patching, and optimization.
 - **Cost efficiency** by scaling automatically and avoiding idle resource costs.
- **Pro SQL Warehouses**

- Supports Photon and Predictive IO, but not Intelligent Workload Management.
- Use when serverless is unavailable, or when custom networking is required (for example, federation or hybrid/on-premises connections).
- **Classic SQL Warehouses**
 - Compute resources run in your **own Azure subscription**, not in Databricks.
 - Takes approximately 4 minutes to start and scales with less responsiveness than serverless.
 - Supports only Photon; no Predictive IO or Intelligent Workload Management.
 - Use only for basic interactive exploration when serverless or pro aren't options.

MLflow

MLflow is an open-source platform designed to manage the **end-to-end machine learning (ML) lifecycle**. It helps data scientists and ML engineers track experiments, manage models, and streamline the process of moving models from development to production. MLflow also supports generative AI workflows and includes tools for evaluating and improving AI agents.

All units complete:

[< Previous](#)[Complete module](#)